

Creación y Gestión de Imágenes despliegue local y en la nube

Sesión 2 · Tu software, empacado y listo para viajar

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida

Docente: Jawy Andrés Romero Pinto

MSc · Especialista en ciberseguridad y nube híbrida · LFCE

De la Sesión 1 a hoy

El salto: de consumir imágenes a producirlas

La semana pasada (S1)

- Corriste contenedores desde imágenes DE OTROS (nginx, ubuntu, hello-world).
- Comprobaste que el contenedor es efímero.
- Aprendiste: imagen = receta · contenedor = plato servido.

Hoy (S2)

- **Escribes TU receta: el Dockerfile.**
- Construyes TU imagen y la versionas con tags.
- La publicas en Docker Hub y la corres desde OTRA máquina.
-  Y la endureces: contenedor sin root.

Entregable de hoy: tu propia imagen publicada en Docker Hub — visible desde cualquier lugar del mundo.

Objetivos de la sesión

Al final de estas 2 horas podrás...

Escribir

un Dockerfile: la receta en texto plano que define tu imagen, paso a paso.

Construir

imágenes con docker build, entendiendo capas, caché y etiquetas de versión.

Publicar

tu imagen en Docker Hub y ejecutarla desde cualquier otra máquina con un pull.

Endurecer

tu imagen: usuario no-root, imagen base mínima y cero secretos adentro.

¿Qué es un Dockerfile?

Un archivo de TEXTO PLANO con las instrucciones, en orden, para construir tu imagen: desde qué base parte, qué archivos copia, qué instala y qué comando ejecuta al arrancar.

Por qué texto plano importa

- Se versiona en Git como el código
- Cualquiera del equipo lo lee y lo audita
- Reconstruible: misma receta → misma imagen
- Documentación viva del ambiente

Lo que reemplaza

- El manual de instalación de 20 páginas
- El “pregúntale al que lo configuró”
- Los servidores artesanales irrepetibles
- Las diferencias entre dev, pruebas y prod

Anatomía de un Dockerfile

Las 6 instrucciones que cubren el 90% de los casos

FROM	La imagen base de la que partes (siempre oficial ).	FROM nginx:alpine
WORKDIR	El directorio de trabajo dentro de la imagen.	WORKDIR /app
COPY	Copia archivos de tu carpeta hacia la imagen.	COPY index.html /usr/share/nginx/html/
RUN	Ejecuta un comando DURANTE la construcción.	RUN apk add --no-cache curl
EXPOSE	Documenta el puerto que usa la aplicación.	EXPOSE 80
CMD	El comando que corre al ARRANCAR el contenedor.	CMD ["nginx", "-g", "daemon off;"]

RUN pasa en el build (una vez); CMD pasa en el run (cada arranque). Es la confusión clásica — grábatela hoy.

Tu primer Dockerfile, completo

El que construirás hoy en el lab — una web servida por nginx

```
# La receta de mi-web · 101-EAN S2

FROM nginx:alpine
# imagen base oficial y mínima (~8 MB)

COPY index.html /usr/share/nginx/html/
# mi página reemplaza la de bienvenida

EXPOSE 80
# documenta el puerto del servidor

# CMD ya viene definido en nginx:alpine
# (arranca el servidor automáticamente)
```

Léelo como una historia:

- 1 Parte de nginx oficial, versión mínima
- 2 Mete MI contenido adentro
- 3 Declara el puerto
- 4 Hereda el arranque de la base

2 instrucciones tuyas bastan: la base oficial ya hizo el trabajo pesado.

La imagen se construye por CAPAS

Y eso hace los builds rápidos y las imágenes eficientes

CAPA 3 · COPY index.html (TU cambio — KBs)

CAPA 2 · configuración de nginx

CAPA 1 · base alpine (~8 MB)

Cada instrucción del Dockerfile = una capa

Lo que ganas con las capas

- Caché: si una capa no cambió, Docker la reutiliza — el rebuild toma segundos.
- Eficiencia: 10 imágenes basadas en alpine comparten UNA sola copia de la base.
- Orden importa: lo que MÁS cambia (tu código) va al FINAL del Dockerfile.
- **Hoy lo verás: el segundo build de tu imagen será casi instantáneo.**

docker build: de la receta a la imagen

Un comando, tres piezas

```
docker build -t mi-web:1.0 .
```

docker build

Construye una imagen leyendo el Dockerfile

-t mi-web:1.0

La eTiqueta: nombre y versión de tu imagen

. (el punto)

El contexto: la carpeta con tu Dockerfile y archivos

La salida del build te muestra cada capa construyéndose — y en el segundo build verás **CACHED** en las que no cambiaron.

Tags: el versionado de tus imágenes

nombre:etiqueta — así se identifica todo en el mundo Docker

```
mi-web:1.0  
local
```

```
jawy402/mi-web:1.0  
en Docker Hub (usuario/)
```

```
registro.ean.edu/mi-web:1.0  
en un registro privado
```

Buenas prácticas

- Versiona SIEMPRE: 1.0, 1.1, 2.0...
- El tag es inmutable en tu flujo: lo que probaste es lo que despliegas
- Usa tags descriptivos para variantes: 1.0-alpine

El peligro de :latest

- latest NO significa “la más reciente” — es solo el tag por defecto
- Hoy latest puede ser una imagen; mañana, otra distinta
-  En producción: tag explícito, nunca latest a ciegas

Publicar tu imagen: el flujo completo

De tu terminal al mundo — lo que harás hoy

¿Por qué el paso 2? En Docker Hub tu imagen vive bajo TU usuario: el nombre completo es usuario/imagen:tag.

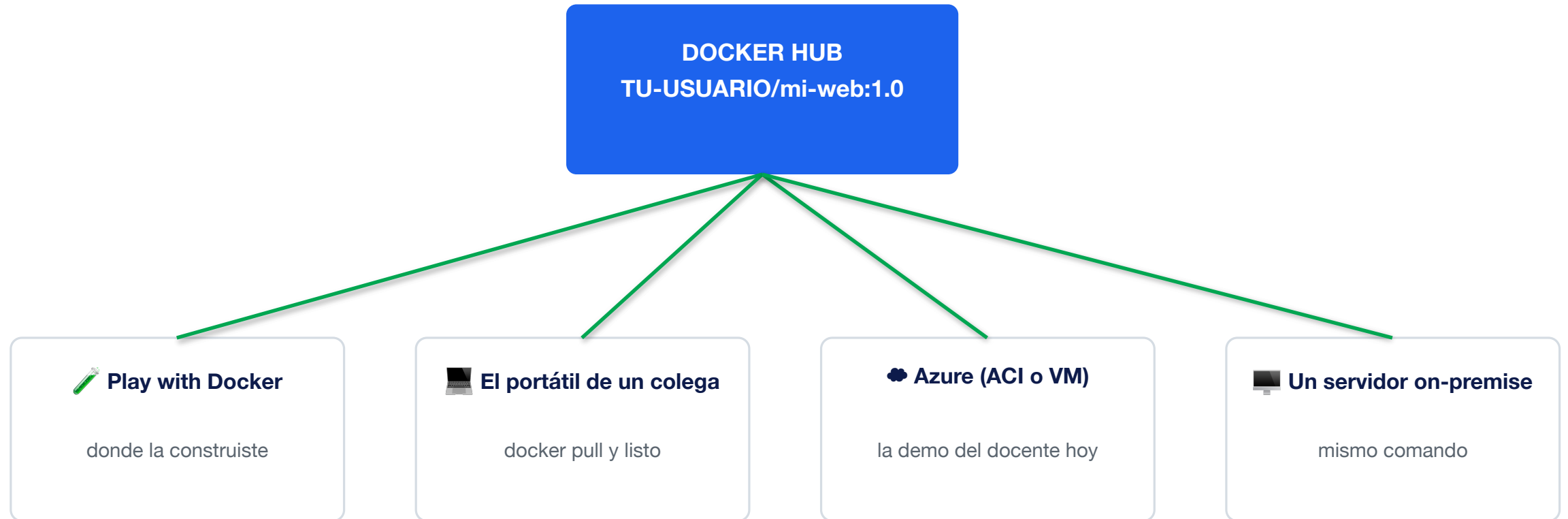


Y el momento de la verdad — en OTRA máquina:

`docker run -d -p 8080:80 TU-USUARIO/mi-web:1.0` → tu web corre en una máquina que jamás vio tu código. Eso es una imagen publicada.

Tu imagen, corriendo en cualquier parte

El mismo pull desde cualquier destino



Publicaste una vez; el mundo hace pull. La portabilidad de la S1 — pero ahora con TU software.

¿Docker Hub o un registro privado?

Dónde deben vivir las imágenes según su naturaleza

Docker Hub (público)

- Gratis, global, perfecto para aprender y para proyectos abiertos.
- Lo que subas en cuenta gratuita es PÚBLICO: cualquiera lo ve y lo baja.
- Neutral: no pertenece a ninguna nube — sirve a todas.
- **Hoy lo usamos para tu primera imagen (sin datos sensibles).**

Registro privado (ACR en Azure)



- Azure Container Registry: solo tu organización ve y usa las imágenes.
- Control de acceso con las identidades corporativas (Entra ID).
- Integrado con ACI, App Service y AKS para desplegar directo.
-  **Para una entidad como Colpensiones: las imágenes internas SIEMPRE en registro privado.**

Regla simple: aprender y demos → Hub público · software institucional → registro privado (ACR).



Eje de seguridad · S2: imágenes que no regalan poder

Tres hábitos desde tu primera imagen

1 · Usuario no-root

Por defecto el proceso corre como root DENTRO del contenedor. Si comprometen la app, el atacante entra con privilegios. Usa variantes unprivileged o define USER en el Dockerfile — hoy lo pruebas en el lab.

2 · Imagen base mínima

alpine (~8 MB) o slim en vez de la imagen completa: menos paquetes = menos vulnerabilidades esperando ser explotadas — y builds y pulls más rápidos.

3 · Cero secretos en la imagen

Nunca contraseñas, tokens o llaves en el Dockerfile o en archivos copiados: la imagen es legible por cualquiera que la tenga (docker history lo muestra todo). Los secretos se inyectan al RUN, no al BUILD — lo veremos en S5.

La seguridad no se agrega al final: se hornea en la imagen desde el Dockerfile.

Buenas prácticas de Dockerfile

Las 5 que separan una imagen amateur de una profesional

Una imagen, una responsabilidad

Un servicio por imagen: la web aparte de la base de datos. (En S3 los conectamos en red.)

Lo que más cambia, al final

El orden de las capas decide el caché: base e instalaciones arriba, tu código abajo.

Base oficial y versionada

FROM nginx:1.27-alpine — nunca FROM de un desconocido, nunca sin versión. 

.dockerignore

Excluye del contexto lo que no debe entrar: .git, node_modules, archivos de datos, secretos.

Imagen pequeña = ataque pequeño

Cada MB de más es superficie de ataque y minutos de despliegue. alpine/slim por defecto.

Los comandos de hoy

Se suman a los 7 de la Sesión 1

docker build -t <nombre:tag> .

Construye la imagen desde el Dockerfile de la carpeta actual.

docker tag <origen> <usuario/nombre:tag>

Crea el alias con tu usuario para poder publicarla.

docker login

Autentica tu terminal contra Docker Hub.

docker push <usuario/nombre:tag>

Sube tu imagen al registro.

docker history <imagen>

Muestra las capas de la imagen — y delata secretos mal puestos. 🔒

docker exec <contenedor> whoami

Con qué usuario corre el proceso: la prueba del no-root. 🔒

El laboratorio de hoy

60 minutos · Play with Docker · tu imagen al mundo

Parte 1 · Construye tu imagen (25 min)

Escribe tu index.html personalizado y tu primer Dockerfile; constrúyelo con docker build, córralo y míralo en el navegador. Luego cambia el HTML y reconstruye: verás el caché en acción con la versión 1.1.

Parte 2 · Publícala en Docker Hub (20 min)

docker login, tag con tu usuario y push. Después, la prueba reina: abre una INSTANCIA NUEVA de Play with Docker (otra máquina) y corre tu imagen con un solo comando.

Parte 3 · Endurécela (15 min)

Comprueba con whoami que tu contenedor corre como root; reconstrúyelo sobre la base unprivileged y comprueba que ya no. Cierra con docker history: las capas no mienten.

Entregable de la S2: la URL pública de TU imagen en Docker Hub + captura del pull en la instancia nueva.

Las 3 ideas que te llevas hoy

1 · El Dockerfile es la receta en texto plano: versionable, auditable y repetible.

2 · Build → tag → push: publicaste una vez y cualquier máquina del mundo hace pull.

3 · La seguridad se hornea en la imagen: no-root, base mínima, cero secretos.

Próxima sesión: conectas varios contenedores entre sí — redes aisladas y una app multi-contenedor.